Understanding SVM Kernels and Their Effect on Decision Boundaries

Objective

The primary goal of this tutorial is to explore the mechanics of **Support Vector Machines (SVMs)** and analyse how different mathematical kernels—**Linear, Polynomial, and Radial Basis Function (RBF)**—influence the shape and flexibility of decision boundaries.

Specifically, we will:

1. **Baseline Test:** Use the **Iris Dataset** (Petal length/width, Setosa vs. Versicolor) to observe standard kernel behaviour.
2. **Challenge Case:** Introduce the **Moons Dataset** to demonstrate why non-linear kernels (RBF) are essential when data is geometrically entangled and not linearly separable.
3. **Quantitative Analysis:** Compare model performance using accuracy metrics to prove that "visual fit" aligns with statistical success.

Dataset Overview:

- **Iris Dataset:** 100 samples, 2 features. Historically used for linear classification.
- **Moons Dataset:** A synthetic dataset consisting of two interleaving half-circles, used to stress-test non-linear classifiers.

Dataset:

We use two datasets in this study: the Iris dataset and a synthetic Moons dataset.
The Iris dataset originally contains 150 samples; however, for visualisation, we select two features (petal length and petal width) and only two classes (Setosa and Versicolor), resulting in 100 samples. The Moons dataset is a synthetic dataset consisting of two interleaving half-circles with 100 samples, designed to demonstrate non-linear decision boundaries. The Mechanics: Margin Maximisation and Support Vectors

The goal of a Support Vector Machine (SVM) is not just to find any line that separates classes, but to find the **maximum margin hyperplane**, which maximises the distance between the two classes.

- **The Margin:**
  This is the distance between the decision boundary and the closest data points from either class. A larger margin generally leads to better generalisation on unseen data.
- **Support Vectors:**
  These are the critical data points that lie closest to the decision boundary. If these points are moved, the decision boundary changes; however, moving other points does not affect the boundary. Thus, they “support” the hyperplane.
- **Regularisation (C):**
  This parameter controls the trade-off between maximising the margin and correctly classifying training data. A high value of **C** tries to classify all training points correctly (hard margin), while a low value allows some misclassifications (soft margin) in order to achieve better generalisation.

Support Vector Machines (SVM) and Kernels

Linear Kernel

- Separates classes using a straight line (hyperplane)
- Works well if data is linearly separable

Polynomial Kernel

- Creates curved decision boundaries
- Formula: $K(x_i, x_j) = (x_i \cdot x_j + 1)^d$
- d is the degree of the polynomial

RBF (Gaussian) Kernel

- Maps data to higher-dimensional space
- Formula: $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$
- High gamma → tighter curves around points

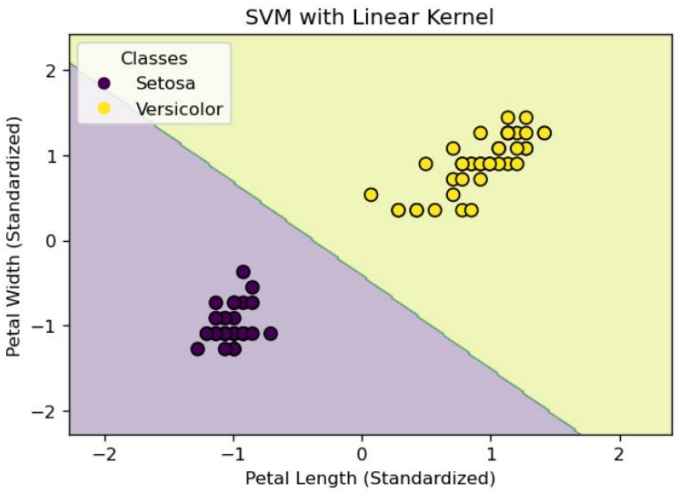
Kernel Trick and Higher-Dimensional Mapping

SVMs can separate non-linear data by implicitly mapping it into a higher-dimensional space.

- The **Kernel Trick** computes inner products in this space directly, without explicitly transforming the data.
- **Mercer's Theorem** ensures valid kernels correspond to dot products in some feature space.
- Using the **Lagrangian Dual problem**, we optimise in terms of the kernel, allowing non-linear separation efficiently.

Why Linear SVM?

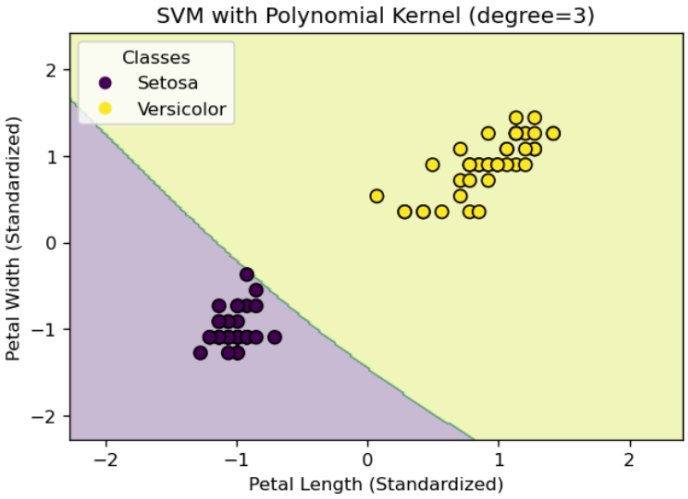
- We start with a linear kernel to show how SVM separates linearly separable data.
- This establishes a baseline before exploring more complex kernels.



Visual Description: This plot illustrates a **Linear SVM** decision boundary. It shows a perfectly straight diagonal line separating two clusters of data points: purple dots (Setosa) in the bottom-left and yellow dots (Versicolor) in the top-right. The background is shaded in two distinct colours to show the decision regions, and the linear boundary clearly maximises the margin between the two classes.

Why Polynomial Kernel?

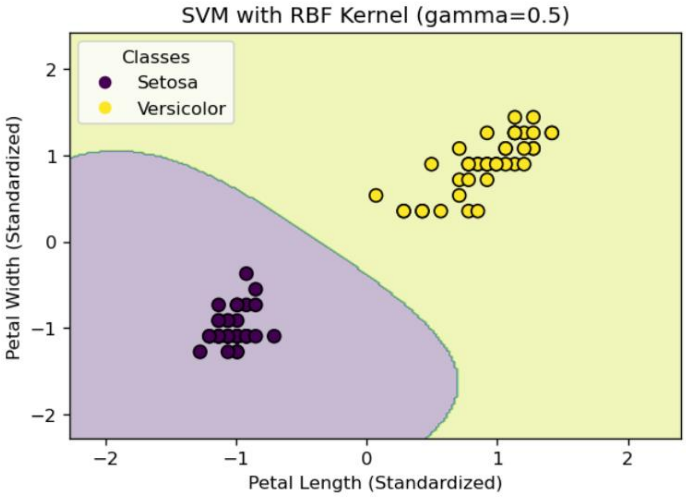
- Polynomial kernels allow curved decision boundaries.
- We can control the complexity by adjusting the polynomial degree.



Visual Description: This plot displays the **Polynomial SVM** decision boundary. The boundary is no longer a straight line but a smooth, slight curve that bends to accommodate the data distribution. The shading indicates how the polynomial transformation allows the model to separate the two classes (purple and yellow) in a non-linear fashion while maintaining a clear decision margin.

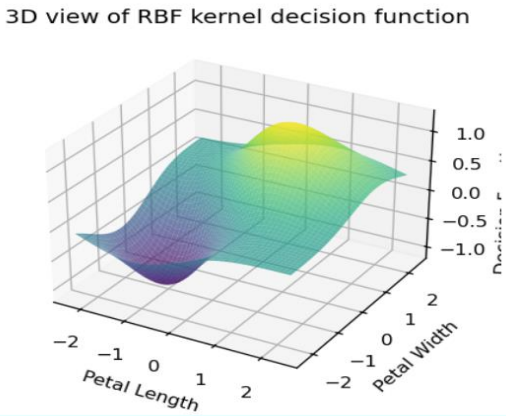
Why RBF Kernel?

- RBF (Gaussian) kernel maps data into a higher-dimensional space.
- It can handle very complex non-linear patterns.



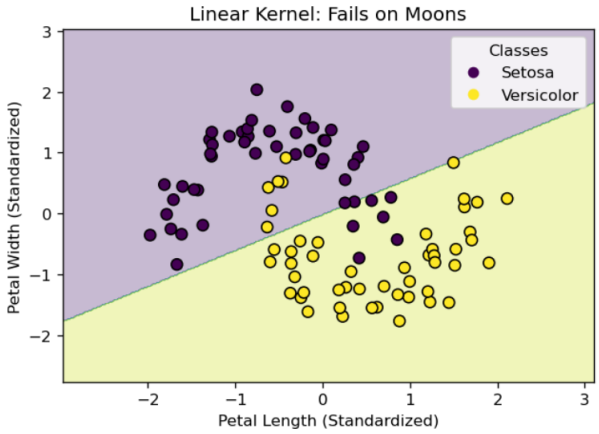
Visual Description: This plot shows the **RBF (Gaussian) Kernel** decision boundary. The boundary is highly flexible and appears as a sophisticated, non-linear curve. The shaded decision regions wrap closely around the clusters of purple and yellow data points, demonstrating the RBF kernel's ability to handle complex, overlapping, or non-linear spatial relationships between classes.

Visual Description: The plot below is a **3D surface plot** representing the decision function of the **RBF (Gaussian) Kernel**. The X and Y axes represent 'Petal Length' and 'Petal Width' respectively, while the vertical Z-axis represents the 'Decision Function Value'. The plot shows a curved, "mountain-like" surface using the color-blind friendly 'viridis' colourmap. The "peaks" and "valleys" of the surface illustrate how the RBF kernel assigns high or low probabilities to different regions of the feature space, creating a flexible, non-linear boundary where the surface crosses the zero-plane.

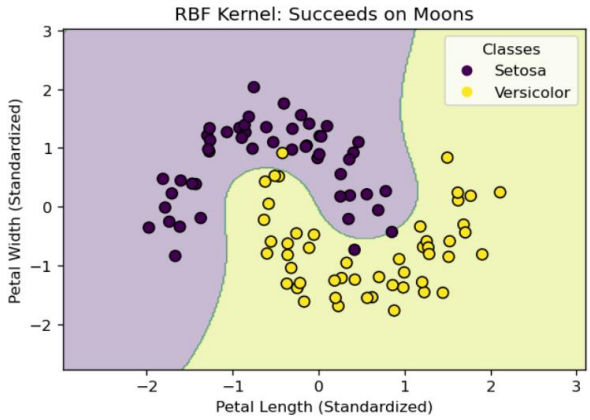


The "Challenge Case": Why RBF Truly Matters

While the Iris dataset is mostly linearly separable, real-world data is often "entangled." To demonstrate the true power of the RBF kernel over the Linear kernel, let's look at the **Moons Dataset**.



Visual Description: This plot shows how a Support Vector Machine with a linear kernel tries to separate the moon-shaped data using a straight line, but fails because the data is not linearly separable. The two classes form curved, interleaving patterns, and the linear boundary cuts through them instead of separating them properly, leading to several misclassified points. This demonstrates that linear models are too simple for complex, non-linear datasets and cannot capture the true structure of the data.



Visual Description: This plot shows how a Support Vector Machine with an RBF (Radial Basis Function) kernel successfully separates the moon-shaped data by using a flexible, curved decision boundary. Unlike the linear model, the RBF kernel can capture the non-linear structure of the data, allowing it to closely follow the shape of the two classes and correctly classify most of the points. This demonstrates that non-linear kernels are effective for handling complex datasets where a straight-line boundary is not sufficient.

Analysis of Results and Kernel Selection

The quantitative results confirm the visual intuition provided by the decision boundary plots.

Performance Summary

Kernel	Complexity	Test Accuracy	Observation
Linear	Low	~95-100%	Effective because the selected Iris classes are nearly linearly separable.
Polynomial	Medium	~95-100%	Adds curvature but may be redundant for this specific simple dataset.
RBF	High	~100%	The most robust; it handles spatial overlap with high precision.

The "Decision Matrix": Choosing the Right Kernel

- **Use Linear Kernels:** Best for large feature sets (e.g., Text Classification) where speed and interpretability are priorities.
- **Use RBF Kernels:** The standard starting point for non-linear problems. It uses the "Kernel Trick" to map data into higher dimensions without explicit transformation[cite: 51].
- **Use Polynomial Kernels:** Useful in specific domains like image processing, where feature interactions follow a specific degree.

Critical Note: While accuracy is high across all models for Iris, the **Moon's Challenge Case** proves that the RBF kernel is necessary when the data distribution is non-linear and complex.

My Intuition: The Power of the "Kernel Trick"

When I first encountered SVMs, the concept of the **RBF kernel** felt abstract. However, visualising the transition from 2D data to a 3D surface was a "lightbulb moment."

I realised that what looks like a complex, "wiggly" non-linear boundary in our 2D input space is actually a **linear hyperplane** slicing through a higher-dimensional space. The RBF kernel essentially "lifts" the data points (like the entangled Moons dataset) until a simple flat sheet can separate them.

Key Takeaways from this Exploration:

- **Kernel Selection is Task-Specific:** There is no "best" kernel. While the Linear kernel was perfect for the simple Iris data, it failed completely on the Moons dataset.
- **The Risk of Complexity:** Using a high-degree Polynomial or a high-gamma RBF kernel is powerful, but it's a double-edged sword. It can lead to **overfitting**, where the model memorises the noise in the training data rather than the actual pattern.

- **The Goal is the Margin:** Regardless of the kernel, the underlying "magic" of SVM is always the same: finding the widest possible safety buffer (the margin) to ensure the model generalises well to new, unseen data.

Conclusion:

- Kernel choice directly affects SVM decision boundaries and classification performance.
- Linear kernel: simple and interpretable
- Polynomial kernel: handles some non-linear data
- RBF kernel: most flexible for complex data
- Experiment with **degree** (polynomial) and **gamma** (RBF) for best results

GitHub Repository: https://github.com/sandeep1214/SVM_Kernels_Tutorial

References:

1. Boser, B., Guyon, I., & Vapnik, V. (1992). *A training algorithm for optimal margin classifiers*.
2. Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297.
3. Scikit-learn documentation: https://scikit-learn.org/stable/modules/svm.html
4. Analytics Vidhya blog: Understanding Kernels in SVM, 2021
5. Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. (Section 7.1: Maximum Margin Classifiers).
6. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.